SASTI DAWAI

Frontend Product Requirements Document

React • Vite • TypeScript • Mobile-first PWA

Version 1.0 • 13 October 2026
Aligned with the main Sasti Dawai PRD and ML PRD

SAFE BY DESIGN • EVIDENCE-BASED • MOBILE-FIRST

Purpose & boundaries

Sasti Dawai's frontend is a mobile-first installable Progressive Web App (PWA) for users who may be elderly, have low digital literacy, or prefer Hindi. It guides them from photographing a medicine strip to checking extracted fields, viewing a verified comparison, estimating savings and finding a Jan Aushadhi Kendra.

Never turn uncertain AI extraction into a confident result. Visibly distinguish needs confirmation, Exact, Close and No verified exact match. Keep “Confirm with your doctor or pharmacist.” visible.

Goals

- One obvious primary action: scan a medicine strip.
- Readable at 360 × 800 and usable on small phones, with large touch targets and clear loading/error states.
- Allow users to inspect and correct extracted information before matching if critical fields are uncertain.
- Show prices with dates, honest match status and savings only for EXACT results.
- Support English/Hindi and patchy networks without pretending an offline scan has completed.

Out of scope

- Diagnosis, dosing advice, treatment decisions, medicine purchase/payment, or automatic switching recommendations.
- Invented prices, equivalence claims, stock status or medical explanations.
- Mandatory login.

Frontend technology stack

Layer	Choice	Reason
Framework	React + Vite + TypeScript	Fast iteration, typed components, client-side app.
Styling	Organized CSS/CSS modules or existing project system	Responsive design without unnecessary coupling.
Components	Accessible reusable primitives; shadcn/ui if it fits current setup	Consistent forms, dialogs and alerts.
API state	TanStack Query or a shared typed fetch client	Loading/error handling and server-state caching.
Forms	React Hook Form + Zod, optional	Typed validation of editable fields.
i18n	react-i18next	English/Hindi; medicine names are not machine-translated.
PWA	vite-plugin-pwa / Workbox	Installability and safe app-shell caching.
Camera	MediaDevices getUserMedia + canvas compression	Camera capture and upload in browser.
Tests	Vitest + React Testing Library; Playwright if time allows	Component and end-to-end behavior tests.

Suggested folder structure

```
frontend/src/app/
frontend/src/components/{layout,feedback,medicine}/
frontend/src/features/{scan,confirm,results,monthly-list,kendras,search,feedback}/
frontend/src/api/{client,schemas,scan,medicines,kendras}.ts
frontend/src/hooks/
frontend/src/i18n/{en,hi}/translation.json
frontend/src/styles/
frontend/src/utils/
frontend/src/types/
frontend/public/
frontend/tests/
frontend/.env.example
```

Information architecture & screens

Screen	Content	Actions/states
S01 Splash/language	App name, promise, language chooser, short privacy note.	Choose English/Hindi; continue offline.
S02 Home	Big Scan button, manual search, monthly list, nearest Kendra.	Scan, search, list, Kendra; offline banner.
S03 Scan	Camera preview/frame, tips, capture/retake, add another side.	Permission-denied gallery fallback; quality warnings and upload progress.
S04 Confirm/edit	Brand, composition, strength, form, release, pack size, MRP, expiry and confidence flags.	Edit fields, retake, add photo, confirm; block unsafe progression.
S05 Result	Exact/Close/None, prices/unit prices, date, savings only for Exact, warning and disclaimer.	Add to list, share, directions, read aloud, report wrong.
S06 Alternatives	Side-by-side ingredient, strength, form, release, route, pack, price, source/date and reason.	Show only backend-returned options.
S07 Monthly list	Items, user-confirmed units/month, item savings and totals.	Edit/remove, duplicate-salt alert, share.
S08 Kendra finder	Map/list, address, distance, known phone/hours, directions, stock caveat.	Use location or PIN/city; handle denied GPS.
S09 Share summary	One-page summary of medicines, verified salts, prices/date and disclaimer.	Download/share; plain-text fallback.
S10 Search	Brand/salt typeahead and typo-tolerant results.	Select result; honest no-results state.
S11 About/data	Data sources, freshness, evaluation summary, privacy and safety.	Open sources and send feedback.
S12 Settings	Language, text size, read-aloud, clear local data.	Apply immediately.

Critical scan journey

- Tap Scan and grant camera access, or upload a gallery image.
- Validate file type/size and run lightweight blur/brightness/glare checks; offer retake guidance for poor images.
- Upload 1–3 compressed images to FastAPI and show honest progress.
- Render extraction and field status. Do not hide low-confidence fields.
- User confirms/edits critical fields and sends them to the backend confirmation endpoint.
- Display only the backend's tier and evidence. Never infer equivalence in the client.
- For EXACT, show backend-verified price/date and savings. For CLOSE/NONE, do not count savings.
- Keep the doctor/pharmacist disclaimer visible on results and share summaries.

Visual design & accessibility

Design rules

- Mobile-first baseline 360 px; usable at 320 px without horizontal scrolling.
- Aim for 44 × 44 CSS-pixel touch targets and 16–18 px body text.
- High contrast; never communicate safety status by color alone.
- Use plain language such as “We couldn't verify this medicine”.
- NTI warnings must not use celebratory green styling.

Accessibility

- Semantic HTML, visible keyboard focus, labelled inputs and meaningful alt text.
- Announce progress/errors accessibly and respect reduced-motion settings.
- Ship Hindi in Devanagari; ensure fonts render it correctly.
- Pair colors/icons with words such as “Please check”. Support zoom and screen readers.

Suggested microcopy

Situation	Copy
Poor photo	“The text is hard to read. Hold steady, move closer, and avoid reflection.”
Low confidence	“Please check the highlighted details before continuing.”
Conflict	“The printed details do not match the catalogue. We cannot verify this yet.”
No exact match	“We couldn't verify an exact lower-cost match. Ask your pharmacist.”
Close result	“This is similar, but it is not an exact match. Ask your doctor or pharmacist before switching.”
Kendra stock	“Stock can vary. Call the Kendra before travelling.”
Disclaimer	“Confirm with your doctor or pharmacist.”

API integration contract

All live medicine decisions come from FastAPI. Configure the API base URL using an environment variable such as `VITE_API_BASE_URL`; never hard-code localhost in production.

- Create one typed API client with timeout/abort handling, request IDs where useful and consistent error parsing.
- Define TypeScript interfaces from the backend Pydantic contracts or generate types from OpenAPI where practical.
- Use multipart/form-data for images; let the browser set the multipart boundary.
- Handle HTTP/network failures as UI states, not uncaught exceptions.

Flow	API
Scan	POST `/v1/scan`
Confirm fields	POST `/v1/scan/{scan_id}/confirm`
Manual search	GET `/v1/search`
Savings	POST `/v1/savings/estimate` or verified match response
Basket summary	POST `/v1/basket/summary` if server-side summary is used; local basket is okay for MVP
Nearby Kendras	GET/POST `/v1/kendras/nearby`
Wrong result	POST `/v1/feedback`
Freshness/about	GET `/v1/meta/data-sources`

Client-side safety rules

- Client-side calculations may support instant display only if the backend contract defines the same formula; final displayed savings must agree with the backend.
- Frontend cannot change CLOSE/NONE to EXACT, infer ingredients, choose a different product or fabricate missing data.
- Use backend-provided source/date fields and show “price as of [date]”.
- Never imply Kendra stock is available without a trusted live-stock source.

Camera, PWA, offline & privacy

Capture

- Use getUserMedia on HTTPS; offer gallery upload if permission is denied or unsupported.
- Allow retake and up to three views of the same pack; explain which side shows brand/composition/price.
- Compress each image to ≤1.5 MB while preserving readable text.
- Show progress and prevent accidental duplicate submissions.

Offline behavior

- Cache app shell, static assets, translations and saved basket; do not indiscriminately cache scan POST responses.
- Explain that a new scan needs connectivity. Never display a fabricated or stale result as live.
- Queued upload is optional; if implemented, show pending status and do not imply a result exists.
- Label cached Kendra/product information with its age.

Privacy

- Do not retain medicine photos locally by default after a completed scan.
- Store the monthly list locally only with clear delete controls; avoid names or health details.
- Request location only after user action; support PIN/city entry when GPS is denied.

State management & error handling

State	Required behavior
Idle	Clear scan/search actions.
Camera permission	Explain permission and offer gallery fallback.
Checking image	Quality check and retake guidance.
Uploading/extracting	Progress and retry/cancel where supported.
Needs confirmation	Field-level flags and editable critical fields.
Verified result	Backend tier, evidence, price date and permitted savings only.
No match	Honest empty state and manual search/pharmacist next step.
Offline/network error	Explain connection issue; preserve safe local list.
API/model error	Recoverable message; never convert failure into a match.
Stale data	Show last-updated date and caution.
Empty list/search	Friendly guidance and one next action.

If two basket entries share a canonical ingredient set, show: “These medicines may contain the same active ingredient. Ask your doctor or pharmacist.” Do not advise stopping either medicine.

Testing & acceptance criteria

Required tests

- Component tests for language switching, field edits, tier labels, disclaimer and text scaling.
- Mock API tests for low confidence, conflicts, Exact/Close/None, null MRP, stale prices and provider failure.
- End-to-end scan on a real Android phone over mobile data; test camera permissions and gallery fallback.
- Responsive checks at 320, 360, 390 and tablet widths.
- Accessibility checks for keyboard, focus, labels, contrast, reduced motion and screen-reader status.
- PWA installation/offline launch; a failed scan must never appear successful.

Definition of done

- User can scan/upload without login.
- Poor images receive actionable retake guidance.
- Uncertain critical fields are highlighted and editable.
- No fabricated results when backend is unavailable.
- Only backend EXACT matches show savings; CLOSE/NONE do not count.
- Prices show freshness date when supplied.
- Disclaimer remains visible on results and share summary.
- English/Hindi toggle works; Hindi is in Devanagari.
- PWA installs and handles offline conditions honestly.

Build order

- Create responsive shell, design tokens, translations, routes and typed API client.
- Build camera/gallery capture, compression, quality tips and request states.
- Build confirmation screen from backend schema, including field edits.
- Build result tiers, warnings, disclaimer and price freshness.
- Add monthly basket, manual search, Kendra finder and feedback.
- Add PWA/offline shell, read-aloud and share/PDF after the core path works.
- Test on a real phone and rehearse backend/model failure paths.

Prioritize a reliable scan → confirm → result flow over adding every stretch feature.

Glossary

Term	Simple meaning
PWA	Website installable and launchable like an app.
React	Library for building interfaces from components.
Vite	Frontend development server/build tool.
TypeScript	JavaScript with types to catch mistakes earlier.
API client	Shared code that sends requests to backend.
JSON	Structured text used to exchange data.
Multipart upload	HTTP format for sending files and fields.
State	Information UI remembers, such as loading or selected language.
i18n	Designing an app to support multiple languages.
Service worker	Browser feature for caching app files and selected offline behavior.
Cache	Local copy used to avoid fetching a resource again.
Accessibility	Making the app usable by people with different abilities.
Responsive design	Layout adapts to screen size.
End-to-end test	Test of a complete user journey.

AI coding-agent instructions

- Inspect the current repository and backend OpenAPI schema first. Do not invent endpoints or response fields.
- Use React + Vite + TypeScript and mobile-first layouts. Reuse the existing styling system rather than adding competing design systems.
- Keep API calls in typed API functions and build all loading, empty, offline, permission, low-confidence, conflict, no-match and error states.
- Never compute medical equivalence in frontend code; treat backend status/tier as authoritative and display it clearly.
- Do not present hard-coded demo data as real. Isolate and label synthetic fixtures.
- Provide .env.example, run/build instructions, tests and acceptance checklist.
- Verify on a real phone or clearly report if device testing was not possible.